

> Ficha Prática Nº10 (React – Criação do Tabuleiro de Jogo)

Na ficha prática Nº9 foi introduzido o conceito de **estado**, com recurso ao hook **useState**, e como este pode ser aplicado em aplicações React. Dando continuidade à implementação do Jogo de Memória em React pretende-se nesta ficha implementar o painel de jogo de acordo com os diferente níveis, sendo ainda introduzido o conceito de um novo hook designado como **useEffect**.

> Preparação do ambiente

- Descompacte o ficheiro **ficha10.zip** e inicie o *Visual Studio Code*.
- Abra a **pasta de trabalho no workspace**.



Os alunos que concluíram a resolução da ficha anterior, devem continuar nesse trabalho.

Os restantes alunos podem resolver esta ficha prática, tendo como base o código fornecido juntamente com esta ficha (Ficha 9).

- No terminal, digite os seguintes comandos:
 - **npm install** (para instalar as dependências em falta)
 - **npm start** (para iniciar a aplicação)
- Confirme a aplicação no browser, no endereço <http://localhost:3000/>

Parte I – Exercício: Criação do Tabuleiro de Jogo

Pretende-se que o aluno implemente o código para apresentação do tabuleiro de jogo de acordo com o nível selecionado, apresentando logotipos aos pares e distribuídos de forma aleatória. No nível básico, devem ser apresentadas 6 cartas, no nível intermédio, 12 cartas e, no nível avançado, 20 cartas. As figuras seguintes apresentam a interface a implementar nesta secção.



Figura 1 - Geração do Tabuleiro de Jogo

1> O primeiro passo, consiste em criar uma estrutura de dados, mais propriamente um *array* de objetos, com a informação das cartas, que serão apresentadas no tabuleiro de jogo. Este *array* será gerado sempre que se seleccionar um nível de jogo. Para implementar este comportamento, poderão ser usadas diferentes abordagens. Neste contexto académico, iremos adicionar código aos componentes **App** e **GamePanel**. Para isso, implemente os seguintes passos:

e. No componente App:

- Declare a variável de estado **cards** com estado inicial []. Esta variável é um array que irá armazenar um *array* de objetos, com as propriedades das cartas que compõem o tabuleiro (o painel de jogo).
- Declare a função **createPanel(level)**, a implementar de seguida, e invoque-a na função **handleLevelChange** existente. Recorde que a função **handleLevelChange**, implementada na ficha anterior, é executada, sempre que é seleccionado um nível.
- A função **createPanel(level)**, que recebe por parâmetro o nível seleccionado, deverá implementar os seguintes passos:
 - > Declare a variável **numOfCardPairs**;
 - > Inicialize a variável anterior, de forma a ficar com o valor de 3, se o nível for '1', com o valor de 6 se o nível for '2' e, por fim, 10 se o nível for '3'. Implemente este comportamento com recurso, por exemplo, a um **switch...case**.
- Adicione, o seguinte código, que, tal como no Jogo de Memória implementado em JavaScript, recorre à função **shuffleArray** que baralha um array, neste caso, o array **CARDS_LOGO**, um array de *strings* (referente aos nomes dos logotipos) e de seguida obtém apenas o número de logotipos necessários perante o nível de jogo seleccionado. Não se esqueça de importar a constante **CARDS_LOGOS** que se encontra definida no ficheiro de constantes, assim como a função **shuffleArray** que se encontra especificada na pasta *helpers*.

```
let initialCards = shuffleArray(CARDS_LOGOS);
initialCards = initialCards.slice(0, numOfCardPairs);
```

- > De seguida, pretende-se gerar um array de objetos **doubledCardsObjects**, em que cada objeto contém as propriedades a serem usadas quando se invocar o componente Card. As propriedades que cada carta terá são:
 - **name** que contém o nome do logotipo a apresentar;
 - **id** que identifica o elemento, como único, e usado depois verificar se é par ou não de outro;

Copie o seguinte bloco de código que implementa o comportamento desejado:

```
const doubledCardsObjects = [];
initialCards.forEach((card) => {
  doubledCardsObjects.push({ id:card, name: card });
  doubledCardsObjects.push({ id: `${card}-clone`, name: card});
});
```

- > De seguida, declare a variável **doubledShuffledCardsObjects** o qual deverá ficar com o array **doubledCardsObjects**, criado no cobaralhado.
 - > Por fim, a função, deverá atualizar a variável de estado **cards** com todos os valores existentes em **doubledShuffledCardsObjects**.
- Visualize o jogo no browser e confirme que não existem erros até ao momento.
- Invoque a função **createPanel**, sempre que o jogo inicia, em **handleGameStart**, de forma a que se gere novo tabuleiro, quando não se altera de nível.
- Uma vez que o componente **Card** é invocado no componente **GamePanel**, a variável de estado **cards** deverá ser passada para esse componente. Assim, altere a invocação do **GamePanel**, adicionando o atributo **cards={cards}**.

f. No componente **GamePanel:**

- Receba a propriedade **cards** (props);
- Altere a invocação do componente **Card**, tendo em consideração que a prop **cards** é um array de objectos, cada um com os dados de uma carta, e na invocação do componente Card deverá ser invocado com a **Key** e **name**.



***Key** é o atributo é usada pelo React para conseguir identificar quando um determinado componente, criado através de uma lista dinâmica, foi alterado, adicionado ou removido. Assim, este mecanismo de identificação, recorre à key e deve ser usado sempre que se criam várias instâncias do mesmo componente, como irmãos, através de uma lista gerada dinamicamente, e, como tal, a posição desses componentes pode alterar.*

- Confirme o desejado no browser, e confirme que as cartas apresentadas são diferentes sempre que inicia ou para o jogo.

2> Como deve ter verificado, o aspeto do tabuleiro não se encontra com o mesmo aspeto que o apresentado na Figura 1, quando se selecciona o nível intermédio/avançado. Isso acontece porque falta adicionar a classe "intermedio" ou classe "avancado" ao elemento elemento `<div id=game>`, para distribuir de forma diferente os elementos. Para isso, implemente os seguintes passos:

a. No componente `GamePanel`:

→ Declare a variável `gameClasse` e será inicializada com uma *string* de acordo com o nível. Assim:

- > Se o nível de jogo for 2, a variável deverá ter o valor "intermedio"
- > Se o nível de jogo for 3, a variável deverá ter o valor "avancado"
- > Senão, deverá ficar sem valor.

→ Especifique o atributo `className` no elemento `div` de acordo com o valor da variável anterior;

b. No componente `App`:

→ Assim, altere a invocação do `GamePanel`, adicionando o atributo `selectedLevel` de forma a que seja passado para este componente, o nível seleccionado.

→ Confirme o desejado no browser, e confirme que as cartas apresentadas são diferentes sempre que inicia ou para o jogo e com a interface desejada.

Parte II – Exercício: Tempo de Jogo

Nesta secção, pretende-se especificar um temporizador para duração do jogo, como apresentado na Figura 2. Existem várias abordagens possíveis para implementar este comportamento, no entanto, será criado um componente `Timer` que irá decrementar o tempo, em segundos, sendo o valor inicial do temporizador obtido através de uma *prop*. Para implementar este comportamento, será introduzido um novo conceito, mais propriamente o *useEffect*.



Figura 2 - Tempo de Jogo

> Hooks: *useEffect*

O Hook **useEffect** adiciona a capacidade de realizar efeitos “colaterais” de forma declarativa, em componentes funcionais, isto é, desencadear um conjunto de ações após renderização de um componente ou alteração do valor das dependências especificadas. Basicamente, quando se invoca este hook, estamos a dizer ao React para executar a função “effect” após surgirem alterações. Assim, o hook auxilia na forma como se pode ligar com os “side-effects” da alteração de um determinado estado. O *useEffect* deve ser declarado dentro do componente, de forma a que tenham acesso aos próprios estados e props. Por omissão, o React executa os efeitos, após cada renderização, incluindo a primeira renderização quando a página é apresentada pela primeira vez. O *useEffect* tem a seguinte estrutura:

```
useEffect(callbackFn, [dependencies]);
```

```
useEffect(() => {
  // código
}, [dependencies]);
```

3> Para implementar o comportamento desejado nesta secção, efetue os seguintes passos:

a. Crie o **componente Timer**, o qual renderiza apenas segundos. Assim, implemente os passos:

- Crie a pasta **timer** na pasta *Components*
- Crie o ficheiro **timer.component.jsx** para o componente e coloque-o **na pasta timer**
- Especifique o *import* das funções *useState* e *useEffect* do React, com a linha de código

```
import { useState, useEffect } from "react";
```
- Efetue o *export default* do componente de forma a ser acessível noutros ficheiros.
- Analise e copie o código da página seguinte que especifica o componente Timer. Após copiar, confirme no browser que o jogo é apresentado e que não existem erros na consola.



*A implementação do timer poderia ser efetuada de outras formas, no entanto, pretende-se que o aluno compreenda conceitos de *useEffect* com a abordagem apresentada, entre outras.*

```
function Timer({ timeout, onTimer }) {
  const [seconds, setSeconds] = useState(timeout);
  const [idInterval, setIdInterval] = useState(null);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds((prevSeconds) => prevSeconds - 1);
    }, 1000);
    setIdInterval(interval);
    return () => clearInterval(interval);
  }, []);

  useEffect(() => {
    onTimer(seconds);
    if (seconds === 0) clearInterval(idInterval);
  }, [seconds, onTimer]);

  return <>{seconds}</>;
}
```

- b. O passo anterior, apenas especifica um novo componente. Agora, é necessário invocar o componente no **ControPanel**. Como pode verificar no código anteriormente apresentado, o componente **Timer**, recebe duas propriedades. A propriedade **timeout**, que especifica o tempo inicial do temporizador e a referência de uma função em **onTimer**. Esta referência, é necessária, para que o **ControlPanel** tenha acesso ao valor dos segundos em qualquer momento, isto é, esta função callback é a forma usada para passar dados do componente filho para pai. Os segundos são necessários no componente **ControlPanel** para, por exemplo, poder terminar o jogo quando chegar ao 0, e outras operações como calcular pontuação. Assim, para invocar o componente timer, implemente os seguintes passos no componente **ControPanel**:

- Especifique a função **handleTimer**, que, quando invocada permite ter acesso ao timer no componente pai.

```
const handleTimer = (t) => {  };
```

- No *return* do **ControlPanel**, invoque o componente **Timer**, no elemento **dd** com **id="gameTime"**, especificando um **timeout** de 10, como apresentado no exemplo.

```
<Timer timeout={10} onTimer={handleTimer} />s
```

- Visualize o jogo no browser, iniciando e parando, várias vezes. Como pode verificar, o timer não voltou a iniciar....

- Para resolver a situação anterior, especifique código para que o componente apenas seja apresentado, quando o jogo tiver iniciado, caso contrário não. Recorra à prop `gameStarted` e operador `&&`.
- Visualize o jogo no browser, iniciando e parando, várias vezes e confirme que o problema ficou resolvido.
- Efetue alterações necessárias, de forma a que o *timeout* varie de acordo com o nível selecionado. Para isso, defina no ficheiro de constantes ou, se já existentes, considere as constantes abaixo:
 - > `TIMEOUTGAME_BASICO = 20`
 - > `TIMEOUTGAME_INTERMEDIIO = 60`
 - > `TIMEOUTGAME_AVANÇADO = 180`
- Por fim, invoque a função `onGameStart` quando o tempo obtido no `handleTimer` for 0. Não deve efetuar qualquer alteração ao componente `Timer`.

Parte III – Exercício para praticar em casa

Implemente o código necessário, para alterar o fundo da cor para vermelho, no elemento onde se encontra o tempo de jogo, quando este chegar aos 10s. Não deve alterar o componente **Timer** e toda a lógica deverá ser implementada no componente **ControlPanel**. As figuras seguintes apresentam o comportamento desejado.



Figura 3 - Timer com fundo vermelho